

Deep Learning-Based Android Malware Detection

A submitted thesis by

Md. Shadhin Mia



Department of Computer Science and Engineering

Pundra University of Science & Technology,

Bogura-5800, Bangladesh

July 29, 2025

Deep Learning-Based Android Malware Detection

A submitted thesis by

Md. Shadhin Mia

ID:0322210105101115

Batch:20th

Under the Supervision of

Iffath Tanjim Moon

Lecturer

Department of Computer Science and Engineering

Pundra University of Science & Technology

Bogura-5800, Bangladesh

July 29, 2025

CERTIFICATION OF ORIGINALITY

We, Md. Shadhin Mia, declare that this submission is the work of all of us and, to the best of our knowledge and belief, it does not contain any material that has been published or written by someone else, nor does it contain any material that has been accepted for the award of any other degree or diploma from a university or other institute of higher learning, except where the text makes clear that it is.

We also say that the ideas in this thesis are the outcome of our work together, even though we may have gotten help from other people with things like style, presentation, and language use.

Md. Shadhin Mia

CERTIFICATION OF APPROVAL

This is to confirm that Md. Shadhin Mia's thesis, "Deep Learning-Based Android Malware Detection," meets the university's rules and standards for originality and quality. It was submitted as part of the requirements for the Degree.

Iffath Tanjim Moon

Lecturer

Department of Computer Science and Engineering
Pundra University of Science & Technology,
Bogura-5800, Bangladesh

ACKNOWLEDGEMENT

We would like to thank our respected supervisor, Iffath Tanjim Moon, and our parents from the bottom of our hearts for their amazing help, support, and encouragement during our thesis project. We are very thankful to our supervisor, Iffath Tanjim Moon, for her amazing knowledge and guidance in the field of Cyber Security. Her unique knowledge and advice have been very important to the achievement of our thesis. Her continual support, helpful advice, and encouragement have been very important in molding our study and making it better. We are very grateful for her guidance, which has given us learning experiences that will stay with us for the rest of our careers. We also want to thank our parents from the bottom of our hearts for always loving, supporting, and encouraging us during our school years. Their constant faith in our abilities, efforts, and understanding has always inspired us. We are very thankful for their unwavering support, which has given us the confidence and motivation to continue our study.

In conclusion, we are honored to thank our wonderful supervisor, Iffath Tanjim Moon, and our dear parents for all the help, support, and encouragement they gave us while we were working on our thesis. Their guidance and constant support have played a big role in our growth and success. We wish them the best in their future successes and health.

Abstract

In this paper, we proposed a deep learning-based methods for detecting Android malwares. A malware is a general-purpose threat to digital devices specifically intended to cause harm and gain unauthorized access to a system. In recent times, there has been an exponential rise of Android malware. In the year 2024, it was estimated that 33.3 million Android devices have been infected with malware and targeted by hackers. Such an application not only interfered with the data of the user, but also deteriorated the system performance and exploited the device permission in order to carry out illegal activities. These days, malware developers are employing new and innovative ways such as obscuring, packing, coding, and encrypting to mislead detection systems and carry out their illegal acts. These techniques that malware developers use to get around the detection system have put traditional malware detectors into the position of not being able to pick up the new variants of malware. Our study focuses on an automatic deep learning approach that is able to learn the differentiating features of the malware pictures. We assessed the efficacy of this methodology with the AndroDEX dataset, a standard image-based collection comprising both benign and malicious specimens. Various deep learning architectures, including CNN, VGG16, ResNet50, MobileNetV2, InceptionV3, VGG19, and DenseNet121, were executed and evaluated by stratified k-fold cross-validation. Performance was evaluated utilizing essential parameters including accuracy, precision, recall, F1-score, AUC, false positive rate (FPR), false negative rate (FNR), and training duration. The results indicated that VGG16 attained the maximum detection accuracy of 95.12%, precision of 94.86%, recall of 95.44, F1 score of 95.14, FPR and FNR are 5.19%, and 4.56% and the highest AUC 98.92%, but CNN offered a favorable equilibrium between speed and performance. The model being able to differentiate between malicious and good apps with such a high accuracy distinctly Demonstrated the potential of our approach.

Key Word: *Android Malware, AndroDEX, Image Data, Android Application, Deep Learning, Cross Validation.*

Table of Contents

ACKNOWLEDGEMENT	iv
Abstract	v
CHAPTER ONE	1
INTRODUCTION	1
1.1 Introduction.....	1
1.2 Contributions.....	2
1.3 Problem Statement.....	2
1.4 Scope of This Research.....	2
1.5 Literature Review.....	3
1.6 Organization of the Paper	6
CHAPTER TWO	7
PRINCIPLE COMPONENTS	7
2.1 Introduction.....	7
2.2 Malware	7
2.3 Deep Learning.....	8
2.4 Android Applications.....	9
2.5 AndroDEX	11
CHAPTER THREE	12
METHODOLOGY	12
3.1 Introduction.....	12

3.2	Proposed method.....	12
3.3	Datasets	13
3.4	Malware Byte Code to Malware Image	14
3.5	Dataset Preprocessing	14
3.6	Proposed Deep Learning Model	16
3.6.1	Convolutional Neural Network (CNN).....	16
3.6.2	VGG16.....	17
3.6.3	ResNet50.....	17
3.6.4	MobileNetV3	18
3.6.5	DenseNet121.....	18
3.6.6	InceptionV3.....	18
3.6.7	VGG19.....	19
3.7	Training Model	19
CHAPTER FOUR.....		21
RESULTS AND PERFORMANCE EVALUATION		21
4.1	Evaluation Metrics	21
4.2	Results and Discussions.....	22
4.3	Experimental results of AndroDEX Dataset.....	23
4.4	Experimental Result Comparison	29
4.5	Comparison with Existing Study	32

CHAPTER FIVE	34
CONCLUSIONS & FUTURE WORK.....	34
5.1 Conclusion	34
5.2 Limitations	35
5.3 Future Work	36
References.....	37

List of Figure

Figure 1:APK File Structure	10
Figure 2: Proposed Methodology.....	12
Figure 3:AndroDEx Dataset Sample Classification	13
Figure 4:Malware Byte to Image Data.....	14
Figure 5:CNN Model Architecture	16
Figure 6:VGG16 Model Architecture.....	17
Figure 7:CNN Model Accuracy Curve on Fold 5.....	23
Figure 8:CNN Model Confusion Matrix on Fold 5	23
Figure 9:CNN Model ROC Curve on Fold 5.....	24
Figure 10:VGG16 Model Confusion Matrix on Fold 5	24
Figure 11:VGG16 Model Accuracy Curve on Fold 5	24
Figure 12:VGG16 Model ROC Curve on Fold 5.....	25
Figure 13:MobileNetV2 Accuracy Curve on Fold 5	25
Figure 14:MobileNetV2 ROC Curve on Fold 5	25
Figure 15:MobileNetV2 Confusion Matrix on Fold 5.....	25
Figure 16:ResNet50 Accuracy Curve on Fold 5.....	26
Figure 17:InceptionV3Model Accuracy Curve on Fold 3	26
Figure 18:InceptionV3Model Confusion Matrix on Fold 3.....	26

Figure 19:ResNet50 Confusion Matrix on Fold 5	26
Figure 20:InceptionV3Model ROC Curve on Fold 3	26
Figure 21:ResNet50 ROC Curve on Fold 5	26
Figure 22:DenseNet121 Accuracy Curve on Fold 2	27
Figure 23:DenseNet121Confusion Matrix on Fold 2	27
Figure 24:DenseNet121 ROC Curve on Fold 2	28
Figure 25:VGG19 Accuracy Curve on Fold 5	28
Figure 26:VGG19 ROC Curve on Fold 5	28
Figure 27:VGG19 Confusion Matrix on Fold 5	28
Figure 28: Proposed DL Model Performance Comparison	30
Figure 29: Proposed DL Model Performance Heatmap	30
Figure 30: Proposed DL Model Training Time Comparison.....	31
Figure 31:Comparison with Proposed DL Model False Positive & False Negative Rate	31

List of Table

Table 1: Riterature Review	5
Table 2:Data Preprocessing Hyperparameter	15
Table 3: Results Comparison Proposed All DL Model	29
Table 4:Performance Comparison of Proposed Model with Existing Study	33

CHAPTER ONE

INTRODUCTION

This chapter introduces the research on deep learning techniques are useful for detecting Android malware. It talks about how mobile risks are getting worse and how traditional ways of finding them don't work as well. The chapter also provides the idea of finding viruses by turning Android apps into pictures. A short summary of the research's main goals and scope is given. Lastly, it gives a brief outline of how the rest of the chapters in this study are set up.

1.1 Introduction

Android is the most used mobile operating system in the world, and more and more people are using it every day. The Google Play Store is an example of a software distribution platform that has a big effect on the fast growth of Android devices and the excitement around the Android app market. However, the inherently open nature of the Android ecosystem also creates an environment conducive to malicious software, putting users' security and privacy at significant risk without their awareness. Consequently, the confusion between harmful software and legitimate applications has become a critical issue at this time [1]. Android Malware is among the most significant threats to user privacy and data security [2]. Android malware is malicious software that was designed to carry out illegal acts on Android users and devices by stealing passwords, releasing private information, installing malicious code, encrypting or deleting files, changing settings, and thus, causing money theft, identity fraud, unauthorized access, and even major trouble in the operations of users. In 2024, there was a significant number of Android malware attacks, There was 33.3 million instances of malware attacks. There were 1.1 million harmful and possibly unwanted installation packages found, with about 69,000 of them being found. This had to do with mobile banking Trojans [3]. Malware exploits vulnerabilities in apps and systems to steal [2]. The rise of Android malware that can pose a severe risk; the risks include data theft, privacy breaches, network disruption, and financial loss. To achieve their goals, cybercriminals use Diverse malware variants, for instance ransomware, Trojans, adware, and spyware, which are often associated with phishing emails, malicious downloads, or compromised websites; these methods serve as the primary means of distribution [4].

Previous studies have explored various techniques for malware detection, including API call analysis, permission-based filtering, and signature-based, but the specified approaches often have significant drawbacks. Current methods don't usually have enough precision at either the static or dynamic analysis levels, which makes mobile device security quite weak. To tackle these problems, an advanced strategy is necessary which not only accurately detects binary malware but also successfully classifies different malware and enhanced Android Security.

1.2 Contributions

This study presents significant advancements in the field of Android malware detection by using deep learning methods and image-based datasets. The main contributions are the creation of strong detection models and thorough analysis to improve accuracy and speed. The contributions are:

1. To develop a deep learning model for detecting Android malware.
2. To develop an image base malware detection technique.
3. To enhance the accuracy of malware detection by leveraging deep learning techniques for static analysis of Android applications.
4. To evaluate the performance of the deep learning model using real-world Android malware datasets.

1.3 Problem Statement

Most traditional methods used to detect malware on Android devices are not effective against new or advanced malware. Because of this, many harmful apps go undetected, putting users at risk of data theft and cyber attacks. To tackle this issue, there is a need for a smart and reliable system that can identify both known and unknown malware accurately. This research focuses on developing a deep learning-based approach to detect Android malware effectively and protect users from these growing threats.

1.4 Scope of This Research

The main focus of this study is to create and test a deep learning-based system that can find malware in Android apps. This study looks into how to use convolutional neural networks (CNNs) and other deep learning architectures to automatically sort Android apps into two

groups: benign and malicious. It does this by looking at both static and behavioral features. The main goal of the research is to look at datasets made up of Android application packages (APKs), change them into forms that are easier to work with, like grayscale photos or feature vectors, and train deep learning models to find patterns that suggest malware. Standard classification metrics including accuracy, precision, recall, F1-score, and ROC-AUC are used to test how well the suggested model works.

1.5 Literature Review

The security of Android applications has become an issue of utmost importance with the exponential rise of Android devices globally. The Android platform, being open source and having an extensive reach, has incidentally increased its vulnerability to malware attacks. The Android malware, which is composed of trojans, spyware, ransomware, and other malicious software, endangers the user's privacy, the integrity of the data, and the device's functionality. This literature study looks at new ways to find Android malware using deep learning, concentrating on models with datasets, their problems, and how they could be improved. The literature review are shown in Table 1.

Santosh K. Smmarwar et al. proposed an to address these problems, this study suggested frameworks based on ensemble learning in combination with advanced feature engineering and meta-heuristic optimization techniques. A framework for Android malware detection was created using Optimized and Efficient Ensemble Learning (OEL-AMD). To ensure that discriminative and informative features were selected, it employed Binary Grey Wolf Optimization (BGWO) to identify the most effective features from both static and dynamic analysis layers. With an accuracy of 83.49% for category classification and 96.95% for binary classification, their combined model beat existing methods in terms of classification performance. Additionally, statistical tests confirmed that the proposed OEL-AMD system shows significant improvements compared to traditional detection methods [5].

Ibrahim et al. Proposed and showed a deep learning model that took into account permissions, services, broadcast receivers, opcode sequences, and also incorporated application size and fuzzy hash as static features. They used the CICMalDroid 2020 dataset and said the results were more than 96% accurate, but this process required a lot of resources[6].

Raval et al. Address machine learning classifiers. Random Forest and XGBoost attain 93% and 92%, respectively. The models demonstrated an ability to differentiate between malware and benign samples, achieving AUC values of 0.93 and 0.92. However, dependence solely on static analysis may restrict the advanced identification of behaviorally dynamic malware [7].

Laya Taheri et al. presents a study-characteristics utilizing dual-layer Android malware analysis programs on the CICInvesAndMal2019 dataset, which encompasses permissions and purposes as static features and API calls as dynamic features. The analysis results demonstrated that the first layer attained 95% accuracy in static-based malware binary classification, while the second layer reached 83.3% accuracy in dynamic-based malware category classification [8].

Vinayakumar et al. proposed a way to use Long Short-Term Memory (LSTM) to find Android malware through both static and dynamic analysis. The experimental findings show that the LSTM did a good job of classifying things in terms of accuracy [9]. Alshahrani et al. showed DDefender is a malware detection system. This method uses both static and dynamic analysis methods, as well as deep neural networks, to find fake Android apps. We use dynamic analysis to get things like system data, network traffic, system calls, and the permissions that are needed. We use static analysis to find the most important parts of a program. We used 2,100 bad apps and 2,100 good apps to train and test the DDefender. The results of the examination show that DDefender was 95% accurate [10].

McDonald et al. proposed a method for finding malware that uses Android manifest permission analysis was suggested, using a static analyzer and APKTool's ability to decompile code for extraction from APKs. We used the AndroZoo repository as the training set for four different machine learning methods. The model validity approach used Random Forest (RF), Support Vector Machine (SVM), Naive Bayes (NB), and K-Means. RF had the highest accuracy, with 82.5% precision and 81.5% recall. Still, this model's accuracy isn't as good as that of other studies in the same field. The reason for this is that the method just looks at the permissions[11].

Table 1: Riterature Review

Ref.	Author	Dataset	Pre- processing Technique	Features Extraction technique	Classifier	Evaluation		
						Accurac y (%)	Precisi on (%)	Reca ll (%)
[5]	Santosh K. Smmarwar at el.	CICInves AndMal2019	z-score normalization technique	BGWO	Optimized Ensemble	83.49	83.36	84.52
[7]	Raval at el.	CICMalDr oid 2020	Application's size, Application's size, Fuzzy Hash	Androguard	GRU	92		
[15]	Aurangzeb et al.	AndroDEX				95	90	92
[12]	Casolar e et al.	Hidden dataset	Tensorflow, GPU	ML	LMT, RandomForest, RandomTree and REPTree	84	83.76	84
[13]	McDonaldt al.	AndroZoo	ML	ML	Random Forest, SVM, Gaussian Naive Bayes, and KMeans	81.53	82.49	85.85
[14]	Almomani et al.	AMD, DREBIN	CNN	CNN	Multi DL Classifier	88.77	87	86

Aldhafferi et al. Introduced a novel malware detection method employing Support Vector Regression with the Radial Basis Function kernel for dynamic feature analysis. The study

found a detection accuracy of 95.74%, a precision of 94.76%, and an AUC of 0.98 in ROC analysis. The outcome unequivocally exhibited superiority compared to the performance of algorithms like SVM, Random Forest, and CNN, while proficiently detecting both known and novel forms of malware. However, dynamic feature extraction may present certain resource limitations, particularly in such an environment [12].

1.6 Organization of the Paper

The organization of the following sections of this follows: Section 2 summarized an in-depth review of the current literature and utilized techniques for detecting Android malware. Section 3 outlines the key principles of this research. The methodology, including dataset details and deep learning models are utilized in section 4. Section 5 provides the evaluation metrics used to measure the performance of the models. Finally, Section 6 outlines the Summary and Future Directions of a research.

CHAPTER TWO

PRINCIPLE COMPONENTS

This chapter introduces the fundamental components that are needed for deep learning to find Android malware. It discusses about what malware is, how Android apps are built, and the deep learning methods used in this study. The chapter also introduces about the malware, android application, deep learning, and datasets. These parts work together to make up the basis for understanding the proposed approach.

2.1 Introduction

In this chapter we give a fundamental description to the main aspects related to this work. and it is used in this study to introduce the reader to the fundamental concepts (i.e. malware, android OS, android applications, deep learning fundamentals and dataset). The comprehension of those components is important for understanding the methods and motivations of the mentioned deep learning based malware detector.

2.2 Malware

Malware is a form of software that gets into our computers without our permission and steals secret data like passwords and bank account numbers. There are many kinds of bad software, or virus are; worms, Trojan horses, ransomware, and spyware are all kinds of malware. It is important for people and businesses to know how each type of malware works and what they can do to stop it. Here, we talk about the popular malwares:

- *Adware*: Adware refers to a software that is made to show annoying ads to users without their permission.
- *Ransomware*: Ransomware is a category of malicious software that obstructs access to our device or its stored files, typically by encrypting them. The assailants subsequently require a ransom payment to regain access. During such attacks, data may be encrypted, erased, or exfiltrated, and occasionally the device itself becomes inoperable. Moreover, cybercriminals may coerce victims by threatening to disclose critical information unless a ransom is paid.

- *Riskware*: This is any legal program that could be dangerous because it has security holes, doesn't work with other software, or breaks the law. Even though these apps weren't made to do bad things, they do have features that could be used for bad things. The Riskware tool could be malware if it is used with bad intentions. Riskware is a very hard threat to deal with because it lives in a hazy area of security.
- *Spyware*: records what the target types on the keyboard, uses the microphone or webcam to spy on them, and sends information about their system. This is done by spyware that changes the protection settings on people's phones. A lot of the time, it comes with legal software. In the used dataset, there are two types of malware: SMS malware and banking malware are malicious software that expropriates personal information from text messages and banking account details from financial applications.
- *Trojans*: Malicious malware that masquerades as legitimate applications, deceiving users into installation.

2.3 Deep Learning

Deep learning is a branch of machine learning, using algorithms and other techniques to model high-level abstractions in data with the help of neural networks with many layers (thus the name), based on the architecture and workings of the human brain. These networks contain artificial neurons that function similarly to real-world neurons. So here is a brief summary of the key concepts and artifacts of what we will encounter during the study of deep learning:

1. *Neural networks*. A neural network essentially consists of several layers that are composed of nodes (artificial neurons), and each layer transforms the data it receives into one that facilitates decision-making or prediction by the network.
2. *Training Deep Networks*: Deep networks use many parameters and many layers; therefore, they require a lot of data to train. During training, the network optimises its weights to reflect the inputs and outputs by using an optimization algorithm, such as gradient descent.
3. *Backpropagation*: Backprop is a training algorithm of neural networks. It estimates the partial derivative of the loss with respect to one of its weights and then adjusts the latter in a way that reduces the loss.

4. *Activation Functions*: These are non-linear functions that give diversity to the network, and they allow it to create complex relationships. They are ReLU (Rectified Linear Unit), Sigmoid, and Tanh. *Convolutional Neural Networks (CNN)*: CNNs are defined to work with grid-structured information such as video or images and learn spatial subdivisions of features without even being trained, and thus they are a breeze to work with images.
5. *Recurrent Neural Networks (RNNs)*: RNNs are awesome at sequence data, time series, natural language, etc. RNN's input to computation has an order, and they maintain a previous state internally. *Generative Models*: They create new data and are similar to data already observed, such as GANs (Generative Adversarial Networks). Algorithms like GANs, as an example, can produce photographs, videos, or even music that realistically appear.
6. *Applications*: Just as the set of tasks to which machine learning has been applied is extensive, so the set of applications to which deep learning has been applied is vast and includes image and speech recognition, natural language processing, autonomous vehicles, and healthcare, where it has been applied in numerous areas such as medical image analysis.

2.4 Android Applications

The Android Package Kit, or APK, extension is used by Android applications. The APK is constructed of a number of components that fall into four categories, and each application may consist of one or more of these types [13]. The four major components of an Android application are:

1. *Activities*: These are the interface modules that deal with user handlings directly. The list of appointments, for instance, would be an activity in an appointment manager application, while a second activity would enable the presentation of information about a single appointment. The various activities are made up of one or more view objects; buttons, labels, etc. that enable user interaction.
2. *Services*: In other cases, they are known as components of service, and they function in the background even when the user navigates to another application. They carry out operations like file downloads, background music, and network transactions.

Additionally, we can make advantage of services that provide inter process communication (IPC) between different apps [14].

3. *Receivers of broadcasts*: When an event occurs on the system as a whole, such as when a device boots up, an SMS message is received, or a phone call is received, these components react and react. They are constantly ready to handle anything the system throws at them since they can do that whether or not the app is closed.
4. *Content suppliers*: These components give the program access to system components and external apps, enabling external code to read or write to the same database that the application uses.

We may construct a complete and functional Android app by putting these components together with the right APIs. Most Android apps are programmed in either Java or Kotlin and then built into an Android package (APK) file that contains data and resource files [6]. The APK file structure are shown in Figure 1. The following are parts of the APK:

1. An XML `AndroidManifest.xml` file with insight like an app sketch and a list of components (like activities, rights, etc.).
2. The `"/META-INF"` folder has the app's certificate, a list of files, a SHA-1 hash, and other items.
3. `Resources.arsc` is a asset file that has been built.
4. A section called `"/lib"` for ready-made code.
5. There should be at least one `Classes.dex` file, which is a Dalvik executable file that operates within its own Dalvik Virtual Machine, or Android RunTime in more recent Android versions.
6. A section called `"/res"` for indexed files like pictures, icons, music, and more.

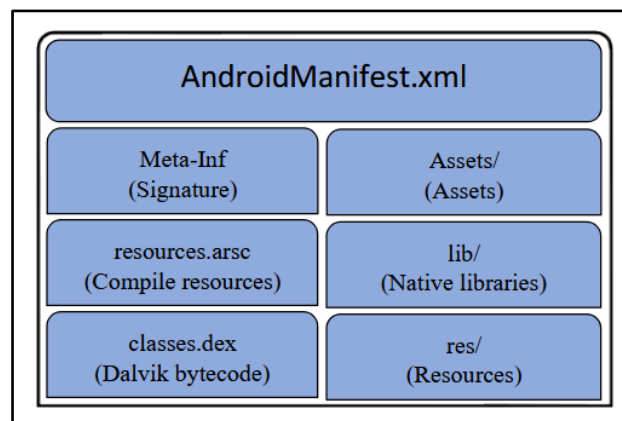


Figure 1:APK File Structure

2.5 AndroDEX

In this study, we utilised the AndroDEX dataset [15]. The AndroDex dataset is an extensive compilation intended for Android malware research, emphasizing static analysis via visual representations. The dataset includes 24,746 samples from more than 180 malware families, as well as benign and obfuscated variants, produced using tools such as AVPass and Obfuscated APK.

CHAPTER THREE

METHODOLOGY

This chapter discuss about the overall framework and techniques employed to develop an impactful Android malware detection system. It discussed about how to turn malware bytecode into images and how to use deep learning models to sort them. The experimental procedure is explained in depth for each phase, such as data preprocessing, model selection, training, and evaluation. The purpose is to make sure that the research can be repeated and to explain the design choices that were taken.

3.1 Introduction

In this research, we compared the experimental result obtained with CNN, VGG16, MobileNetV2, ResNet50, InceptionV3, and DenseNet121 deep learning algorithm and demonstrate deep learning base malware binary classification using static analysis technique.

3.2 Proposed method

In this Framework the proposed methodology overcomes the constraints detecting the android malware using advanced deep learning algorithm technique. This approach will be developed to efficiently evaluate and classify Android apps in a manner that accurately detects and identifies malware. The framework guarantees strong detection against both known and unknown threats and offers extensive insights into various types of Android malware using the AndroDEX datasets. A workflow overview is shown in Figure 2.

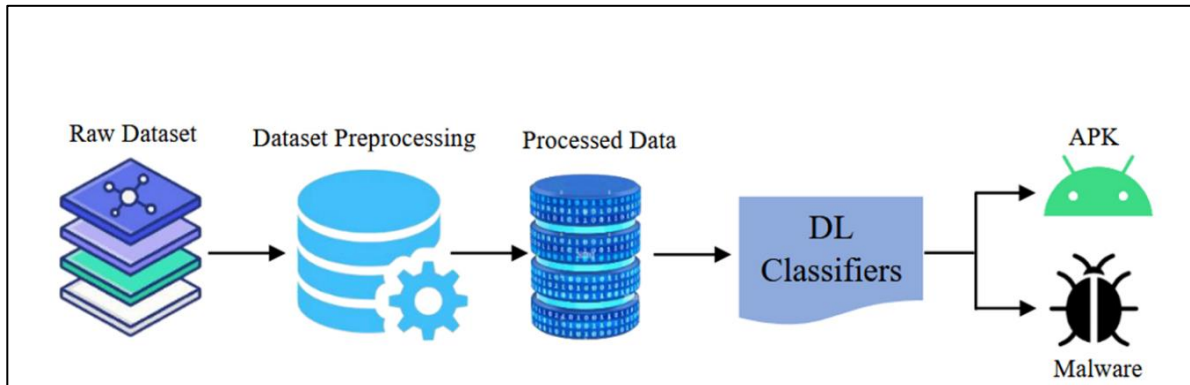


Figure 2: Proposed Methodology

3.3 Datasets

The AndroDEX dataset [15] is The AndroDEX dataset is a prominent benchmark for the static analysis of Android malware and has been extensively utilized in research to evaluate the efficacy of malware detection techniques. It comprises an extensive assortment of Android programs, encompassing both benign and malicious APKs, therefore offering a suitable foundation for the assessment of malware detection frameworks that is both realistic and varied. It is especially appropriate for the scalability and accuracy assessment of static analysis-based detection systems.

The AndroDEX collection comprises 4,415 malware samples distributed across 180 distinct malware families, encompassing various attack paths and forms of destructive behavior. These families exhibit many methodologies for assaults on Android devices, including privilege escalation, SMS fraud, adware, and data exfiltration. This data collection comprises 4,415 benign applications acquired from various sources. In this research we split the dataset into two classes one benign and another is malware. we divided the dataset 80% for train and 20% for testing.

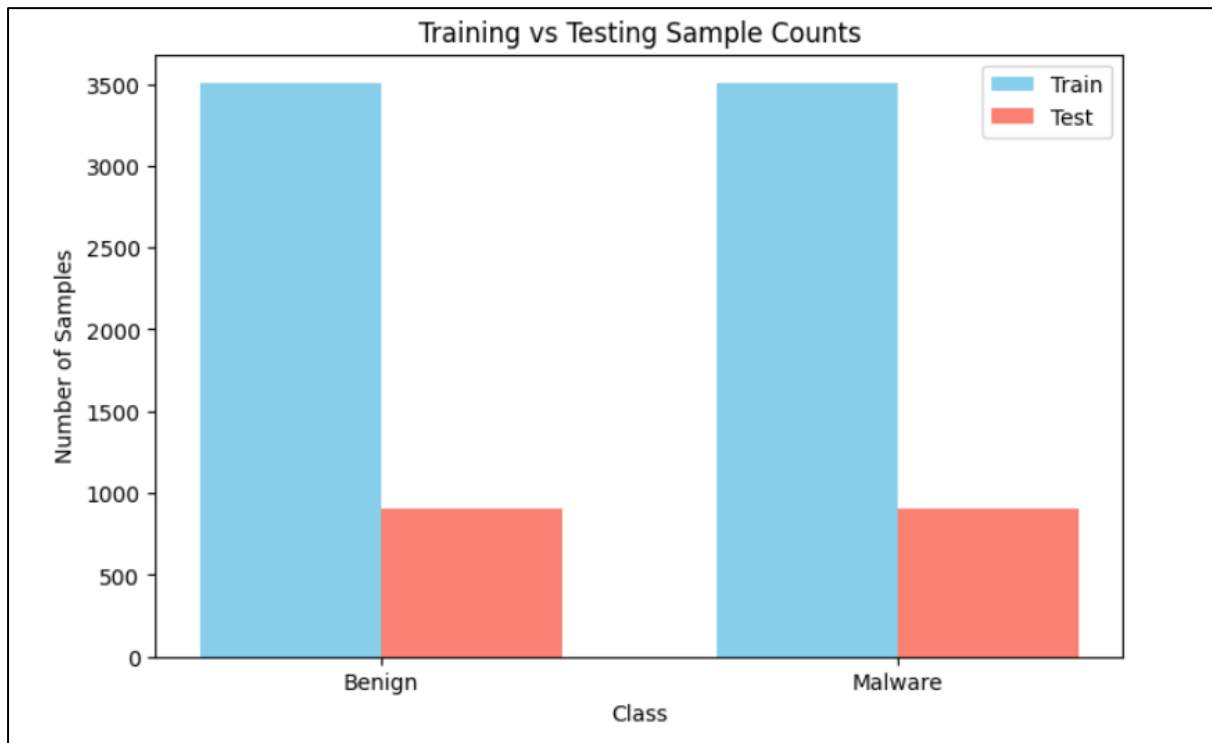


Figure 3: AndroDEX Dataset Sample Classification

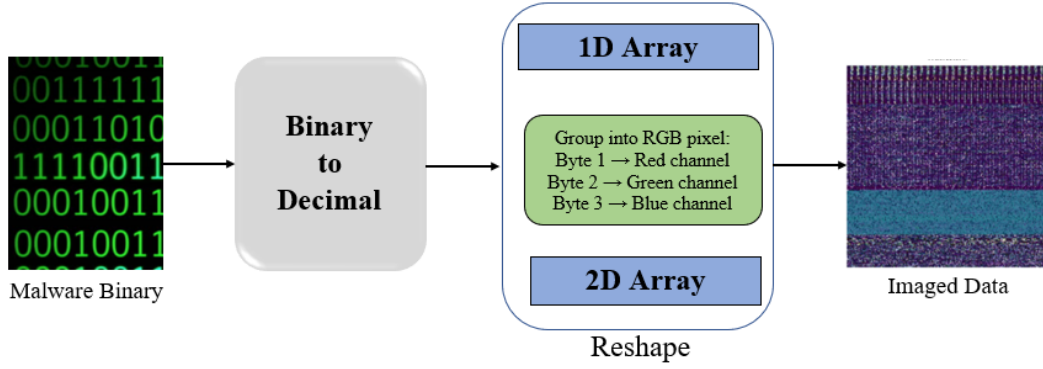


Figure 4: Malware Byte to Image Data

3.4 Malware Byte Code to Malware Image

shows how Converter changes numbers by using a malware scenario. To turn a malware binary into an RGB image, Converter reads the binary data in bytes one at a time, changes each byte into a decimal number between 0 and 255 [28], and then saves the number in a one-dimensional array. For example, the number 0110000 becomes 96. Each integer in the array represents a pixel in an RGB image. A grayscale image's resolution has width and height dimensions, but different malware samples have varying file sizes. This means that one should think about this issue to accept a range of width and height options. Put together every three bytes to make one RGB pixel: The first byte is the red channel, second byte is green channel, and Byte 3 goes to the blue channel. We use a simple method to change the shape of the image data (array) by following the recommended fixed-width with a variable height based on its file size. This is then changed into a 2-Dimensional grayscale image that fits the portable network graphics (.jpg extension) format that are shown in Figure 4 using a built-in function in the cv2 library or PIL library, such as `imwrite` or `imsave`. Table shows the suggested fixed picture widths for different sizes of malware files.

3.5 Dataset Preprocessing

The preprocessing of AndroDEX data is an important stage in order to prepare image-based Android malware and benign samples for the input of deep learning models. As the dataset consists of images of benign and malware applications, the first preprocessing shown in Table 2: Data Preprocessing Hyperparameter step was to resize them all into a fixed shape

Table 2: Data Preprocessing Hyperparameter

Model	Hyperparameters
CNN	Image Size: 128×128 Batch Size: 32 Epochs: 30 Preprocessing: Rescale (1./255) Augmentation: Rotation(15°), Width/Height Shift(0.1), Zoom(0.1), Horizontal Flip, Fill Mode='nearest' Optimizer: Adam, LR = 1e-4
MobileNetV2	Image Size: 224×224 Batch Size: 32 Epochs: 30 Preprocessing: Rescale (1./255) Augmentation: Rotation(20°), Width/Height Shift(0.2), Zoom(0.2), Shear(0.2), Horizontal Flip, Fill Mode='nearest' Optimizer: Adam, LR = 1e-4
VGG19	Image Size: 160×160 Batch Size: 16 Epochs: 20 Preprocessing: preprocess_input (VGG) Augmentation: Rotation(25°), Shift(0.2), Shear(0.2), Zoom(0.3), Brightness[0.8,1.2], Flip, Fill Mode='nearest' Optimizer: Adam, LR = 1e-5
ResNet50	Image Size: 224×224 Batch Size: 16 Epochs: 30 Preprocessing: preprocess_input (ResNet) Augmentation: Rotation(20°), Shift(0.2), Shear(0.2), Zoom(0.3), Brightness[0.8,1.2], Channel Shift(20), Flip Optimizer: Adam + ExponentialDecay LR
InceptionV3	Image Size: 299×299 Batch Size: 4 Epochs: 30 Preprocessing: Rescale (1./255) Augmentation: Rotation(10°), Shift(0.1), Zoom(0.1), Shear(0.1), Horizontal Flip, Fill Mode='nearest' Optimizer: Adam, LR = 1e-5
DenseNet121	Image Size: 224×224 Batch Size: 16 Epochs: 70 Preprocessing: preprocess_input (DenseNet) Augmentation: Rotation(15°), Shift(0.2), Zoom(0.2), Shear(0.3), Brightness[0.7,1.3], Flip, Fill Mode='nearest' Optimizer: Adam, LR = 1e-6

that matches the model's architecture. CNN models take input in 128×128 image size and MobileNetV2 and ResNet50 require 224×224 pixels. InceptionV3, on the other hand has a different architecture and accepts input in 299×299 pixels. This transformation is done to establish a uniform size across the dataset as well as to facilitate neural network compatibility. The image pixels are then normalized after resizing. For models that do not utilise pre-trained weights, such as the custom CNN or MobileNetV2, normalization is performed by scaling pixel values to be in the range [0, 1] by dividing by 255. In contrast, pretrained models (e.g., VGG19, ResNet50, DenseNet121) require that input pixels are processed in a specific way, where the pixel values are adjusted to the same range by some fixed convention, such as normalization or standardization of each channel with specific statistics during training (e.g., mean removal). We will encode class labels ('benign' and 'malware') as 0 and 1 respectively to make it a binary classification problem. This is required to set up the right rounding of the target values for the models during training. To further enrich the dataset and to prevent overfitting, data

augmentation strategies are used during the training. Those are geometric transformation such as rotation, width shift, height shift, shear, and zoom. Models use brighten and shift channel to create all kinds of light. A horizontal flipping is applied too to add variety, and the fill mode is set as 'nearest' to fill the empty spaces in the transformed data. All the images are loaded via a custom function that basically goes through the directory structure using glob, reads in the image files, applies transformations on them, etc. The last processed data is written in NumPy arrays which will be employed for training, validation and testing the deep learning models later. This preprocessing pipeline is organized and strict, but it first prepares the input data to be clean, normalized, and augmented with the best results of a malware detection task.

3.6 Proposed Deep Learning Model

The proposed models are made up of diverse popular architectures, which range from an own-made CNN model to pre-trained models such as VGG16, VGG19, MobileNetV2, ResNet50, InceptionV3 and DenseNet121. Each model has merits, some are directly designed to be computationally efficient, while others are designed for precision in deep image analysis. Through testing a range of structures, we intend to select a strongest model for the waste classification task with superior levels of classification accuracy and robustness on various data repositories.

3.6.1 Convolutional Neural Network (CNN)

The CNN architecture proposed is specifically implemented to process 128x128x3 images. It begins with three convolutional layers followed by batch normalization, and max-pooling layer. The first block consists of 3×3 kernels convolutional blocks of 32 filters, followed by batch normalization and a max-pooling layer with a pool size of (2, 2). A dropout of 0.25 is

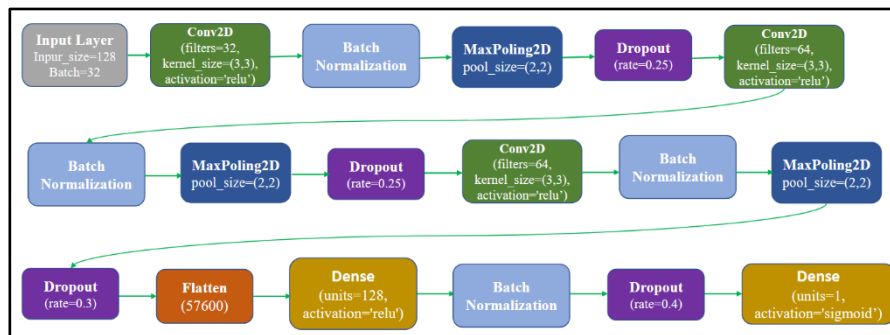


Figure 5: CNN Model Architecture

employed after each block to avoid overfitting. The second and third layers of the convolutional blocks use 64 and 128 filters, respectively, with a convolutional size of 3×3 . The model flattens the output after the convolutional layers and sends it to two dense layers. The first FC layer has 128 units with ReLU activation, and the output layer uses sigmoid activation for the binary classification. The model is trained using the Adam optimizer with learning rate $1e^{-4}$, binary cross-entropy loss, and accuracy metric. The architecture are shown in Figure 5.

3.6.2 VGG16

The VGG16 model uses the pre-trained weights of the ImageNet dataset without its top classification layers shown in Figure 6. The last 4 layers of the base VGG16 model are fine-tuned in this model and the others are frozen to keep the pre-trained features. Following the output of the base model, a global average pooling layer is utilized, followed by batch normalization and dropout as a network regularizer. There are 128 units in the first dense layer with ReLU activation, and the output layer uses a sigmoid activation for binary classification. The training algorithm for this model is Adam with learning rate $1e^{-5}$, and binary cross-entropy as loss (evaluated with accuracy).

3.6.3 ResNet50

The ResNet50 model also uses ImageNet pre-trained weights, but places its emphasis on using the ResNet-style residual connections. The model is based on the ResNet50 architecture without the top classification layers. Only the last 30 layers of the model are fine-tuned and the

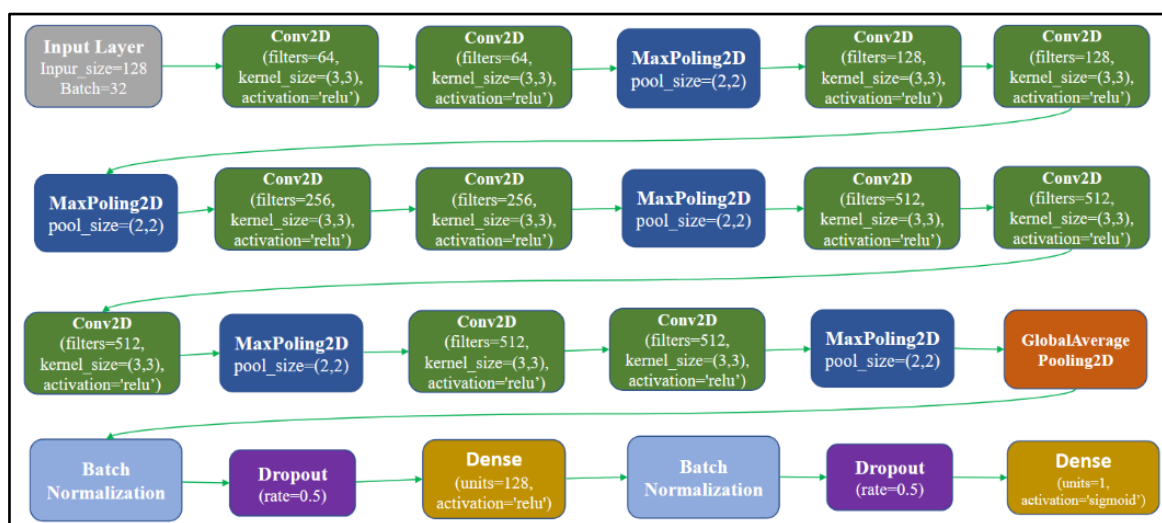


Figure 6: VGG16 Model Architecture

other layers are frozen. The output of the base model is stacked with a global average pooling layer and batch normalization, dropout, and dense layer of 256 units with ReLU activation. The output layer is activated by sigmoid for binary classification. We compile the model with the Adam optimizer with a learning rate scheduler, and binary cross-entropy is set as the loss function while the accuracy is used as model evaluation.

3.6.4 MobileNetV3

The proposed classifier is a MobileNetV3 model using ImageNet pretrained weights for a Mobile optimized performance. The base network used is MobileNetV3 with no top classification layers, and the last layers are fine-tuned and the rest are frozen. On top of the base model output, a global average pooling layer, batch normalization, as well as dropout layer are put in place consecutively. First dense layer has 128 units with ReLU activation and output layer is a sigmoid function for binary classification. The model is compiled with Adam optimizer with learning rate $1e^{-4}$, binary cross-entropy loss, and accuracy as evaluation metric.

3.6.5 DenseNet121

The DenseNet121 model is constructed based on a densely connected convolutional network, in which each layer is a feedforward connected to every other layer. The network adopts the pre-trained DenseNet121 base with the top classification layers removed. The last 100 layers are trained by fine-tuning and the other layers are frozen. After the output of base model, global average pooling layer is added, then followed by three batch normalization and dropout layers to avoid the overfitting. The first dense layer has 256 units, with batch normalization and a dropout layer. The output layer has sigmoid activation for binary classification. The model is built using Adam optimizer with learning rate of $1e^{-6}$, and the binary cross-entropy as loss function, as well as accuracy as the evaluation metric.

3.6.6 InceptionV3

The InceptionV3 model is well-established deep convolutional architecture that is highly efficient for large-scale datasets. We utilize the pre-trained InceptionV3 model without its top classification layers and tune the last layers while keeping the others to be freeze. The output of the base model is first forwarded from a global average pooling layer and then batch

normalization is performed using a dropout layer. The first dense layer of 128 units with ReLU activation function, with the sigmoid output layer for binary classification. The model is compiled using the Adam optimizer with a learning rate of $1e^{-5}$, binary cross-entropy as the loss function and accuracy on evaluation.

3.6.7 VGG19

VGG19 uses the same structure as VGG16, except it has 19 weights layers. The model takes pre-trained VGG19 weights with ImageNet and discards the top classification layers. Fine-tuning is performed to the last 4 layers of the pre-trained VGG19 model and the remaining layers are frozen. After the base model average pooling layer is added, batch normalization and dropout layers follow. The first dense layer contains 128 units with ReLU activation and the output layer applies a sigmoid activation function for binary classification. Adam optimizer with $1e^{-5}$ learning rate is used to compile this model with binary cross-entropy as the loss function and accuracy as the evaluation metric.

3.7 Training Model

We utilized a several of deep learning models, such as CNN, VGG16, MobileNetV2, ResNet50, InceptionV3, and DenseNet121, and VGG19 to train on the AndroDEX dataset. All models were trained using Stratified K-Fold cross-validation for 5 and 3 folds for most models and InceptionV3 respectively. The pivotal hyperparameters were adjusted to get the best performance and the image size we set for CNN is 128x128, 224x224 for most models, and 299x299 for InceptionV3. We use an initial learning rate of 10^{-4} which is divided by ten when the validation error is not improved in 10 epochs. The batch size was set to 32 in most settings, except for DenseNet121 and VGG19 that required batchsize 16. We trained for 30 epochs, except for 20 epochs in the case of VGG19 and 70 for DenseNet121. We used Adam optimizer with learning rate between $1e^{-4}$ and $1e^{-6}$ depending on the model. To avoid overfitting, early stopping, reduce of learning rate ReduceLROnPlateau, and model checkpointing were used as callbacks.

Augmentation like random rotations, width shift, height shift, zooming and horizontal flip were utilized to increase the variation of the training set. For each model, pretrained ImageNet weights are used with only the last few layers finetuned. The performance was measured using several metrics, including accuracy, precision, recall, F1 score, AUC, FPR, or FNR. The

performances of the models were evaluated visually through the confusion matrix, ROC curves, and accuracy plots to compare and evaluate the classifying ability of each model on the dataset.

CHAPTER FOUR

RESULTS AND PERFORMANCE EVALUATION

This chapter shows the results of the experiments and rates how well the suggested deep learning models work for finding Android malware. It has a complete analysis based on important metrics including the accuracy, precision, recall, F1-score, AUC, and false positive rate. We talk about how well models like ResNet50, VGG16, MobileNetV2, and CNN work by comparing them to each other. The results show that the image-based detection method utilized in this study is strong and reliable.

4.1 Evaluation Metrics

We use all the performance metrics to estimate the effectiveness of our proposed deep learning-based approach for detecting android malware based on the AndroDEX Dataset. These performance metrics are obtained based on the attribute the values derived from the training and testing processes of dataset [16]. The meaning of these parameters is described below:

True Positive (TP): Correct classification of an android malware detection. False Positive (FP): Normal traffic is mistaking for an Attack traffic. True Negative (TN): Normal traffic that is classified right. False Negative (FN): Misclassification of an Attack as legitimate Traffic. This study also employs Accuracy (A) to evaluate the stacked model. Besides Accuracy, Precision, Recall and F1-score are also presented [17]. In the area of android malware detection, these three signs are commonly used.

Accuracy- It is the ability of the classifiers to correctly identify the malware signatures from the training set is described as Accuracy.

Equation 1: Calculating Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision- It is the ratio of the number of the positive predictions that are correct to the total number of predictions.

Equation 2: Calculating Precision

$$Precision = \frac{TP}{TP + FP}$$

Recall-The proportion of the data instances detected and classified by a Deep learning model correctly so as to identify each malware is the Recall, which is also known as True Positive Rate (TPR).

Equation 3: Calculating Recall

$$Recall = \frac{TP}{TP + FN}$$

F1 Score- The F1 Score is a compound measure that consists of both precision and recall and is considered to be the most important metric in the domain of malicious malware detection.

Equation 4: Calculating F1-Score

$$F1\ Score = 2 \cdot \frac{Precision \times Recall}{Precision + Recall}$$

4.2 Results and Discussions

In this section We show the results and analysis of our proposed deep learning-based model on AndroDEX datasets. For the model's performance assessment, we performed android malware detection using most important performance metrics such as accuracy, precision, recall, F1-score, confusion matrices, AUC, ROC curve, the training and the validation accuracy plots. The confusion matrix gives a more detailed classification performance of the model including the true and false predictions of malicious and benign samples [18]. The overall performance of the model can be understood by the metrics such as accuracy, precision, recall and F1-score. Accuracy reflects a model's general classification performance, whereas precision and recall reflect the extent to which the model can minimize false negatives and false positives. The F1-score is the harmonic mean of precision and the recall, and it provides a balanced perspective on the model performance. The training and testing accuracy plots show the learning behavior of the model during training, which gives ideas on how well it generalizes to the unseen data. Trend consistency of training and testing accuracies shows lack of over fitting and proper learning. The training ROC curve reflects the optimization process for the model; it shows how

the model reduces the error over epochs. A decrease in the loss such that it decreases is gradual can make sure that the convergence is valid for model to perform better in the detecting android malware.

4.3 Experimental results of AndroDEX Dataset

The proposed model demonstrates excellent performance on the AndroDEX dataset. The k-fold cross-validation was applied for the testing performance of the deep learning models based on the AndroDEX dataset. Each model was then evaluated using standard classification metrics such as accuracy, precision, recall, f1-score, AUC, FPR, FNR, training time. The models were a CNN custom-built, VGG16, MobileNetV2, ResNet50, InceptionV3, DenseNet121, and VGG19. Here is a comprehensive overview of how each model performed.

Our custom CNN model performed well average accuracy 94.47%, Precision 95.12%, Recall 93.78% and F1-score 94.41%. Class separability was excellent AUC 98.59%. Furthermore, it had a low FPR of 4.85% and FNR of 6.22% that (shown in Figure 7, Figure 8, and Figure 9) is crucial for minimizing both false bloodhounds and missed malware detections. One of the strengths of the model was its speed as it was the fastest to train with an average training time of 91.22 seconds, allowing for rapid deployment wherever fast real-time analysis is of the essence. Although the CNN is a shallow architecture (Figure 5) when compared with pre-trained models, it was very effective in the malware classification problem in image-based data.

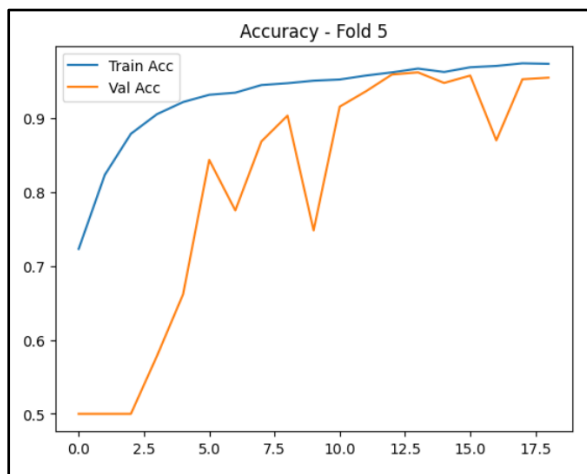


Figure 7: CNN Model Accuracy Curve on Fold 5

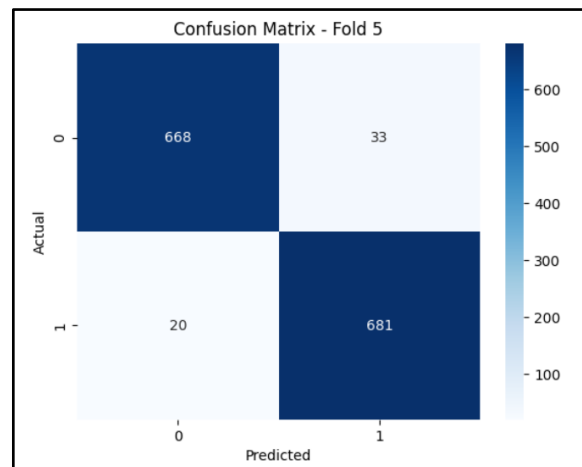


Figure 8: CNN Model Confusion Matrix on Fold 5

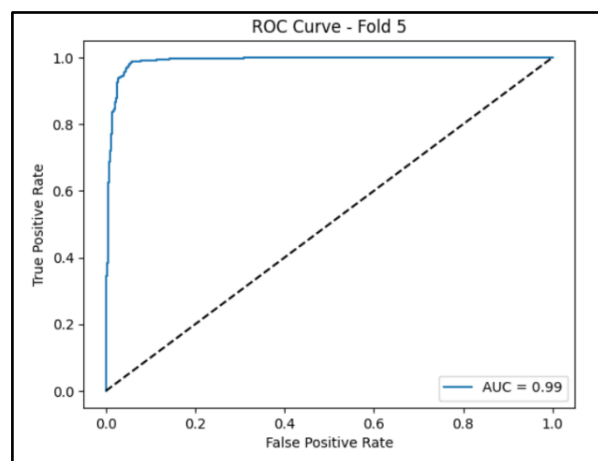


Figure 9: CNN Model ROC Curve on Fold 5

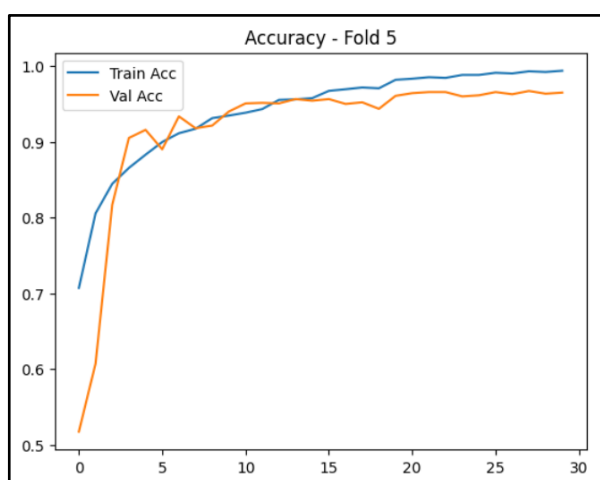


Figure 11: VGG16 Model Accuracy Curve on Fold 5

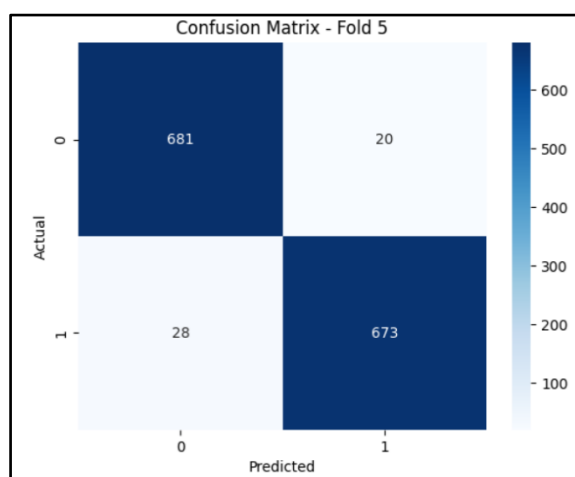


Figure 10: VGG16 Model Confusion Matrix on Fold 5

VGG16 was found to be the best model based on the predictive power and resilience. It produced the best average accuracy of 95.12% (Figure 11) at the precision, recall, and F1-score of 94.86%, 95.44%, and 95.14%, respectively. The highest AUC from drug-like compound was 98.92%, which showed (Figure 10) it was better than others tested and it could be used in distinguishing malware from benign.

In addition, its FPR and FNR, which were both approximately 5.00% (Figure 12), were not high. For this performance, the training time was however paid the model needed 1115.47 seconds to be trained on average in each fold. Such combination of high accuracy and low

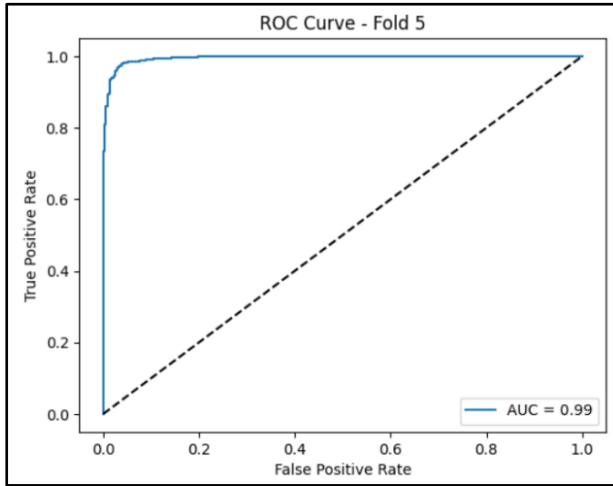


Figure 12:VGG16 Model ROC Curve on Fold 5

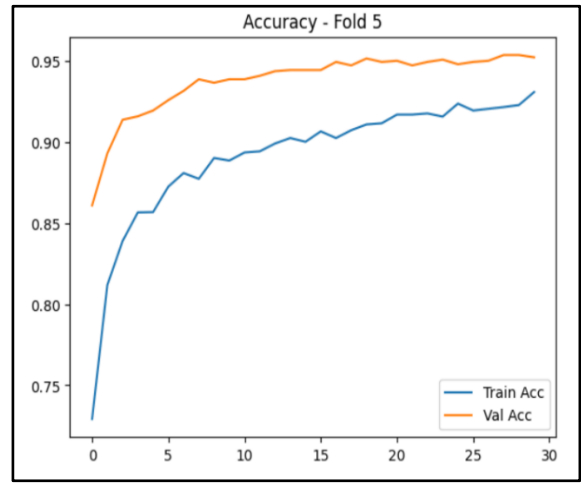


Figure 13:MobileNetV2 Accuracy Curve on Fold 5

error rates makes the VGG16 a strong candidate for practical uses, especially in those systems where predictive performance is considered over execution time.

For the trade-off between efficiency and effectiveness, we chose the MobileNetV2 model. It remained with an average accuracy of 93.31% (Figure 13), precision of 91.83% and recall of 95.09%, a good F1-score of 93.42% and an AUC of 98.02% (Figure 14). It also had small FNR and FPR rates, with 4.91% and 8.47% (Figure 15) respectively. MobileNetV2 received much faster training within 239.29 seconds, evidently shorter than VGG16 and other heavyweight models. These findings indicate that MobileNetV2 is well suited for resource limited setting such as mobile or edge devices where computing resources are limited.

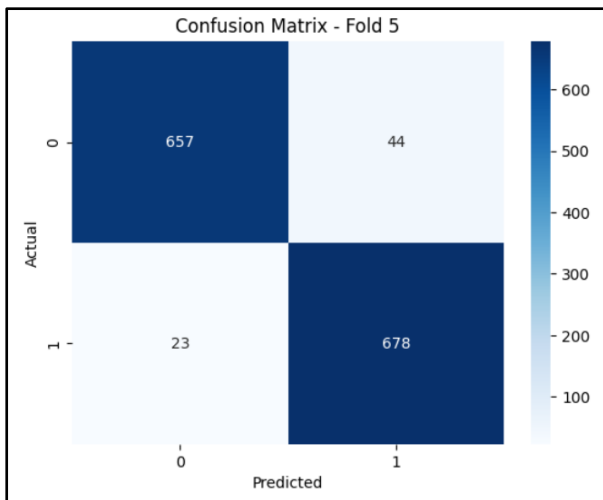


Figure 15:MobileNetV2 Confusion Matrix on Fold 5

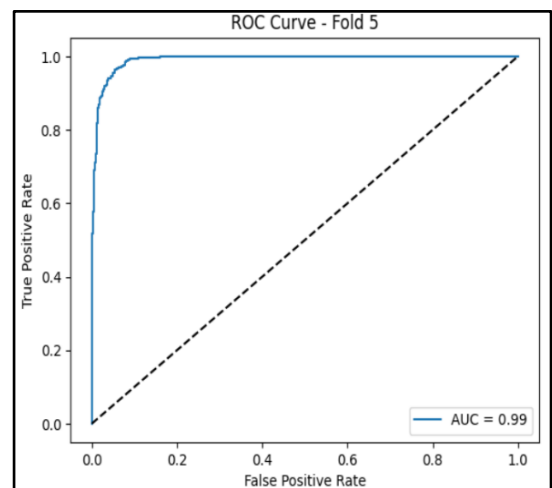


Figure 14:MobileNetV2 ROC Curve on Fold 5

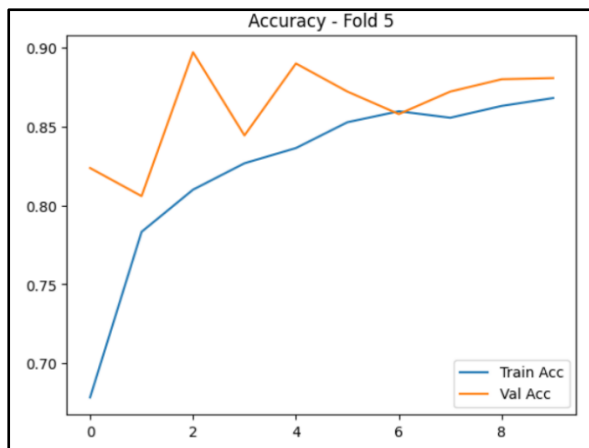


Figure 16:ResNet50 Accuracy Curve on Fold 5

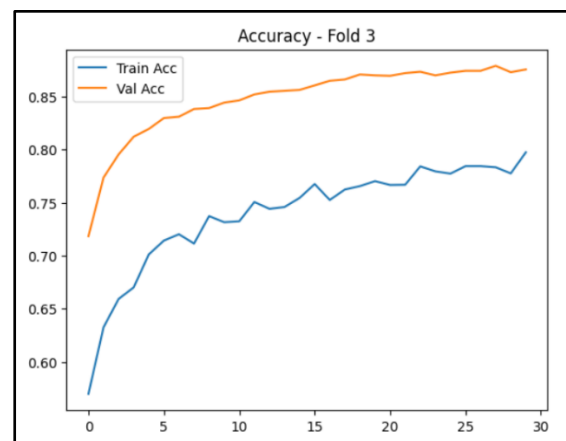


Figure 17:InceptionV3Model Accuracy Curve on Fold 3

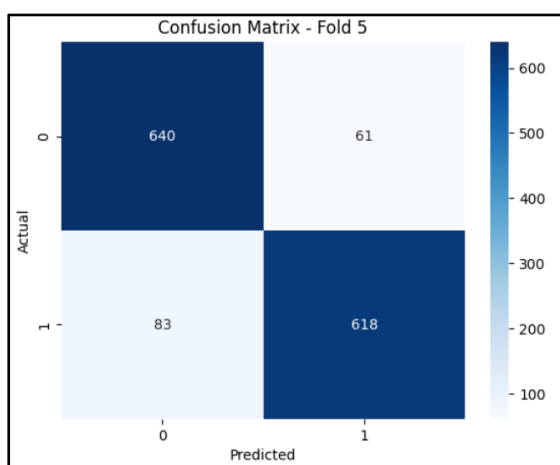


Figure 19:ResNet50 Confusion Matrix on Fold 5

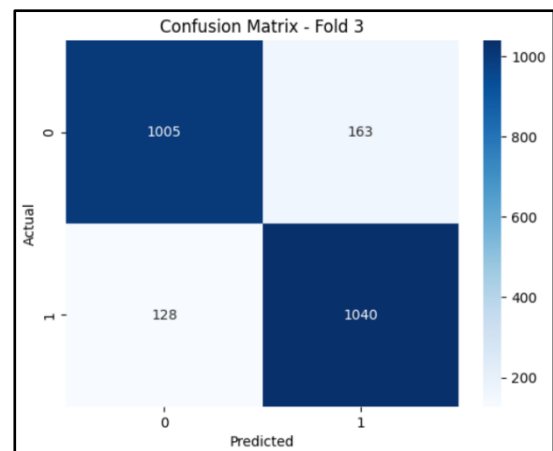


Figure 18:InceptionV3Model Confusion Matrix on Fold 3

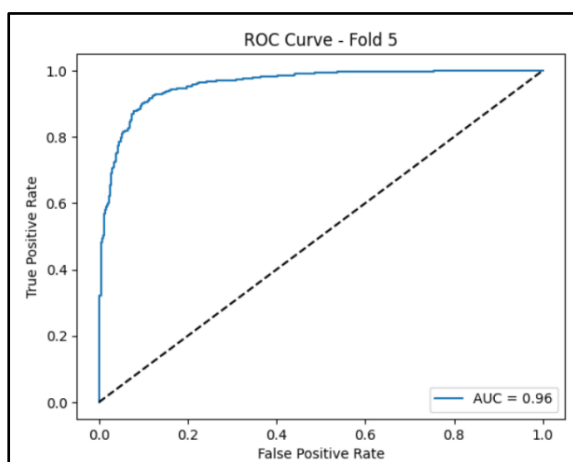


Figure 21:ResNet50 ROC Curve on Fold 5

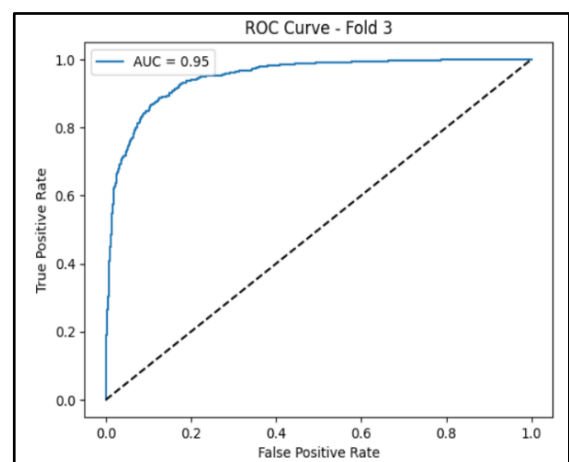


Figure 20:InceptionV3Model ROC Curve on Fold 3

On the other hand, the ResNet50 network, being much deeper and good at classification of images, was not superior to the three much lighter networks in this work. It obtained an accuracy and precision of 89.59% (Figure 16) and 89.65% respectively while and the recall got 89.56%, and the F1-score 89.57%, with the AUC of 96.42% (Figure 21) respectively. Its FPR and FNR were higher than the CNN and MobileNetV2, 10.38% and 10.44%, respectively (Figure 19). The model was also computationally expensive having high average training time of 1508.12 seconds. Although ResNet50 performed reasonably well, its length of training and sensitivity to incorrect predictions may be a drawback in situations where speed and a low error rate are important.

The performance of the InceptionV3 model was fair, with an average accuracy, precision, recall, and F1-score of 86.92%, 85.00%, 89.70%, and 87.28%, (Figure 17) respectively. It achieved an AUC of 94.69% (Figure 20) but FPR 15.86% and FNR 10.29% (Figure 20) were quite high. The training time of the proposed CNN, although less than DenseNet121 and ResNet50, was 1226.79 seconds on average. These observations suggest that the effectiveness of the InceptionV3 for detecting malware is limited due to the high false positive and false negative rates, which may cause poor system usability and detection performance in real usage.

Results The DenseNet121 model provided good recall 91.84% and F1-score 89.83% and acceptable accuracy and precision values of nearly 89.60% (Figure 22) and 87.90%, respectively. It also indicated an AUC of 96.08% (Figure 24), a FPR of 12.64%, and a FNR

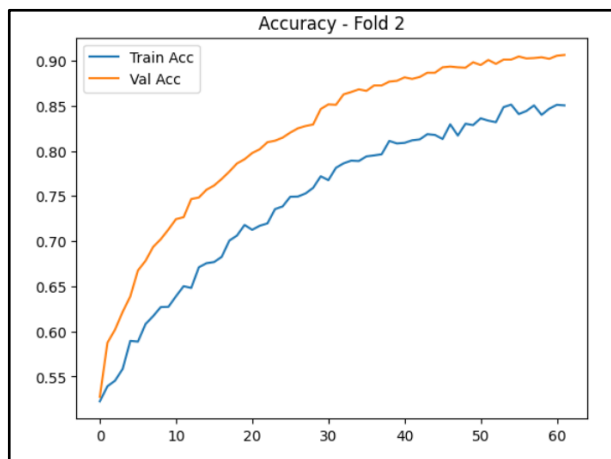


Figure 22:DenseNet121 Accuracy Curve on Fold 2

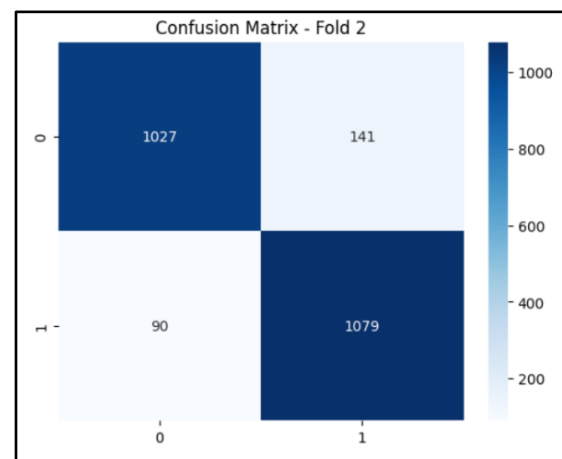


Figure 23:DenseNet121 Confusion Matrix on Fold 2

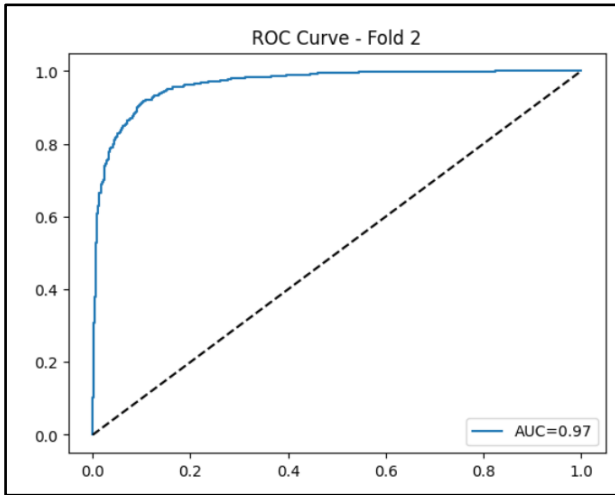


Figure 24:DenseNet121 ROC Curve on Fold 2

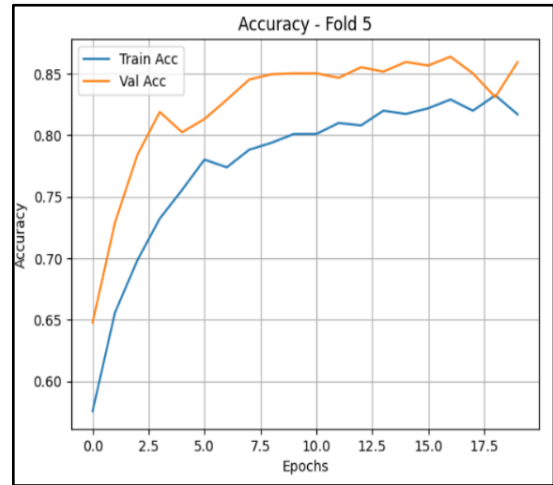


Figure 25:VGG19 Accuracy Curve on Fold 5

of 8.16% (Figure 22). But it has the highest training time so far, with mean time at 4735.57 seconds, which is the critical drawback. Although its deep structure and interconnected layers led to a good classification accuracy, high computational cost of DenseNet121 prevents it from being suitable for learning on-the-go or real-time applications.

Last, VGG19 performed worst among all tested architectures. Its average accuracy of prediction was 81.84% (Figure 25), precision of 78.52%, recall of 89.56%, and F1-score of 83.38%, and its AUC ROC curve (Figure 26) was 91.56%. This was a huge false positive rate of 25.87%, which is a lot of benign apps being incorrectly classified as malware. The FNR was high as well at 10.44% (Figure 27), which indicates that a considerable number of malware instances remained undetected. Although the training time 764.06 seconds was

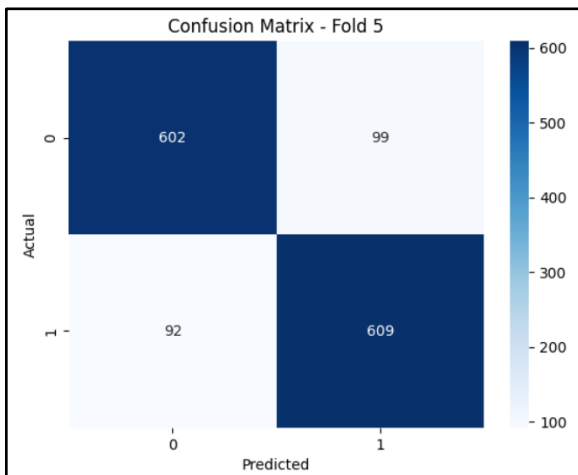


Figure 27:VGG19 Confusion Matrix on Fold 5

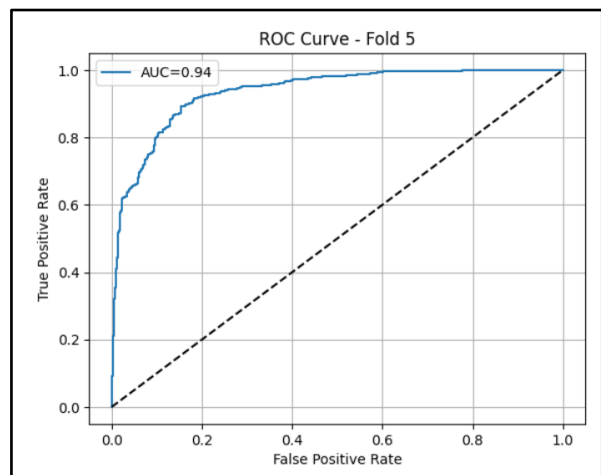


Figure 26:VGG19 ROC Curve on Fold 5

modest, the erratic performances and high error rates demonstrate that VGG19 is not an appropriate candidate for the malware detection problem in this scenario.

In summary, the VGG16 and CNN models were the leading models for the AndroDEX dataset. VGG16 surpassed in accuracy and AUC, and CNN was a good trade-off between speed and accuracy. MobileNetV2 also showed competitive results, particularly suitable for running in resource-limited environments. These findings indicate the necessity to trade off detection rate, training time and computational overhead when choosing a deep learning model for Android malware detection.

4.4 Experimental Result Comparison

Performance of the test results of the different models tested based on several metrics such as Accuracy, Precision, Recall, F1 Score, AUC, false positive rate (FPR), false negative rate (FNR) and Training time: CNN, VGG16, MobileNetV2-2019017, ResNet50, InceptionV3, DenseNet121, VGG19. Of the VGG16 came out best with an average accuracy of 95.12%, having the best precision and recall. It was obtained 98.92% with the highest AUC, denoting the strong classification performance. Nevertheless, it was VGG16 that took the most time to

Table 3: Results Comparison Proposed All DL Model

Model	K Fold Cross Validati on	Accurac y (%)	Precisi on (%)	Recal l (%)	F1 Score (%)	AUC (%)	FP R (%)	FNR (%)	Trainin g Time (s)
CNN	5	94.47	95.12	93.78	94.41	98.59	4.85	6.22	91.22
VGG16	5	95.12	94.86	95.44	95.14	98.92	5.19	4.56	1115.47
MobileNet V2	5	93.31	91.83	95.09	93.42	98.02	8.47	4.91	239.29
ResNet50	5	89.59	89.65	89.56	89.57	96.42	10.38	10.44	1508.12
InceptionV 3	3	86.92	85.00	89.70	87.28	94.60	15.86	10.29	1226.79
DenseNet1 21	3	89.60	87.90	91.84	89.83	96.08	12.64	8.16	4735.57
VGG19	5	81.84	78.52	89.56	83.38	91.56	25.87	10.44	764.06

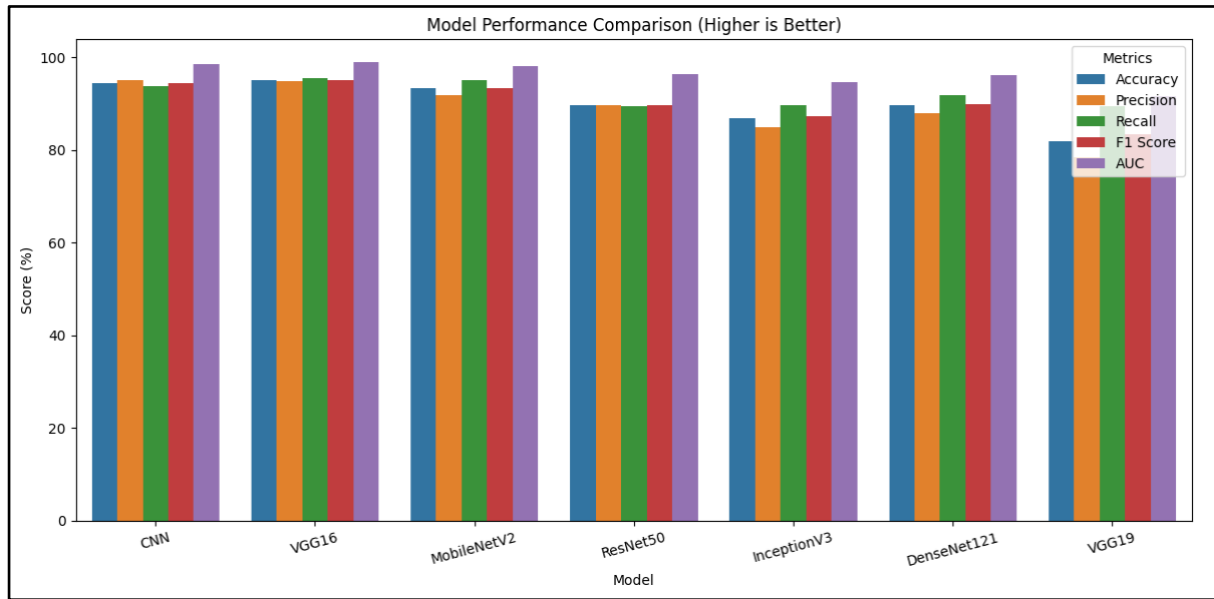


Figure 28: Proposed DL Model Performance Comparison

performing in both sense as the training time is efficient enough to train, averaging at 91.22 seconds. MobileNetV2, the second fastest at 239.29 s, had a lower accuracy with average accuracy 93.31% and decent recall rate 95.09%. The ResNet50, had lowest

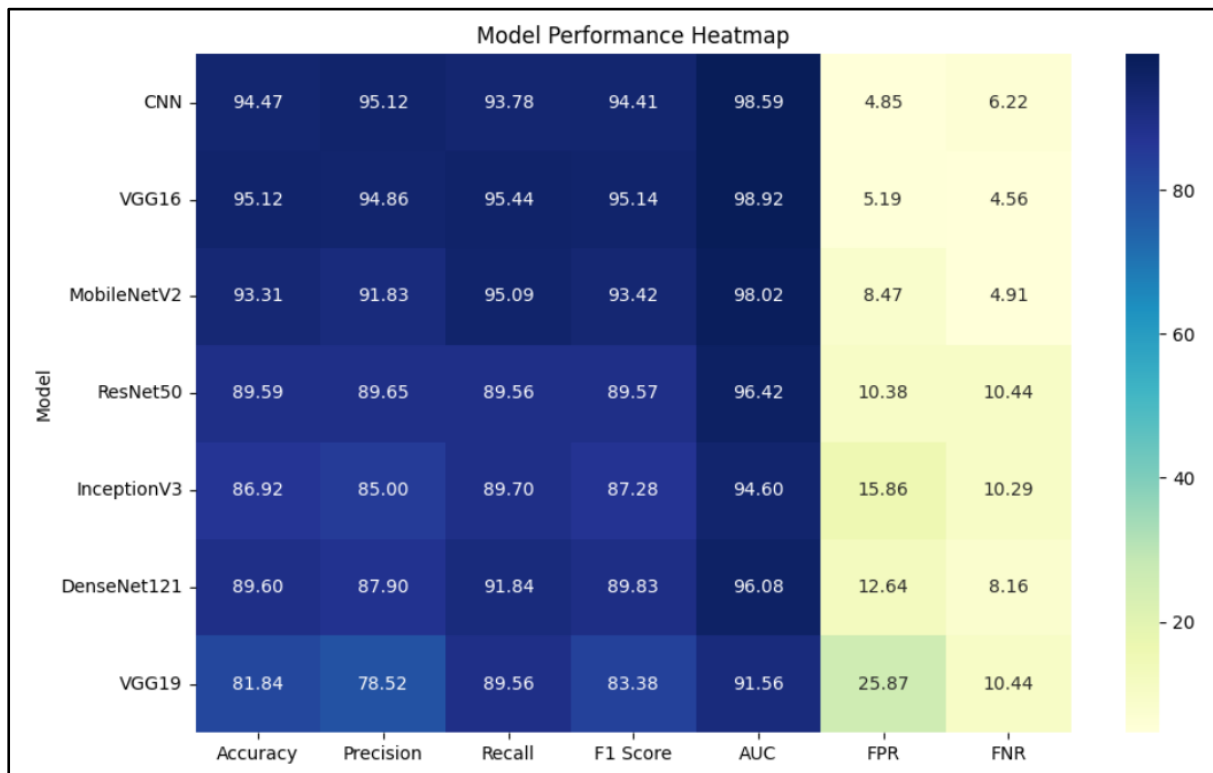


Figure 29: Proposed DL Model Performance Heatmap

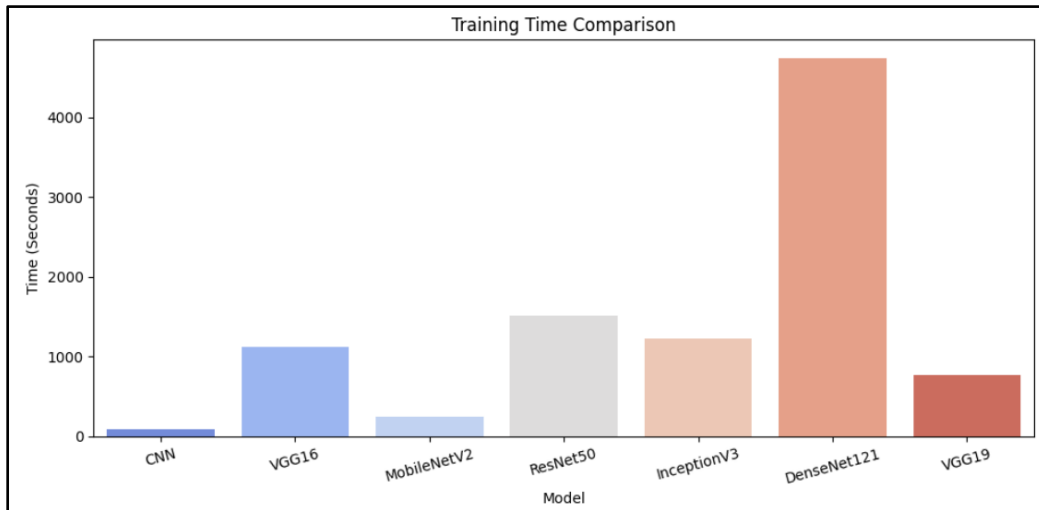


Figure 30: Proposed DL Model Training Time Comparison

model. InceptionV3 performed the worst, having the lowest accuracy 86.92% and AUC score 94.60% which shows that it is poor in classification. Nonetheless, its model training time 1226.79 sec. fell in an average time span compared to the competed models. DenseNet121 achieved good average accuracy 89.60% with a strong recall. It also did well, obtaining an AUC of 96.08%, but had one of the highest rates of false positives at 12.64%. DenseNet121 required the longest time on the training set, with an average of 4735.57 seconds, being the most computationally demanding model among the models compared. Finally, VGG19 had the overall we test performance with the lowest accuracy 81.84% and recall 89.56%. It has the lowest AUC score 91.56% and the highest FPR 25.87%. The only winning algorithm

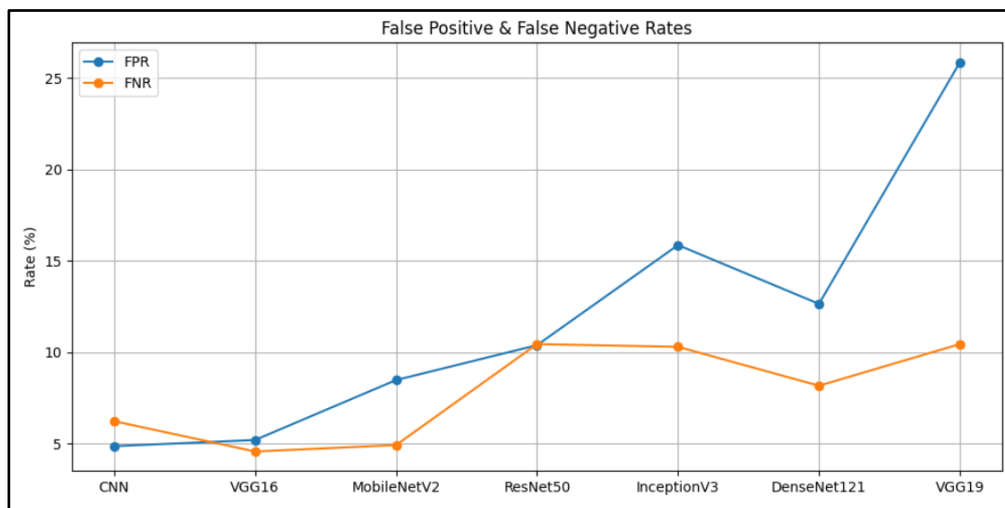


Figure 31: Comparison with Proposed DL Model False Positive & False Negative Rate

produced not particularly positive classification; however, it could be trained in the shortest time of 764.06 s.

4.5 Comparison with Existing Study

We also verify the performance of our proposed approach by comparing our performance measurements with the one found in the previous work on the AndroDex corpus. Our findings confirm the advantage of our proposed method the method in accuracy, precision, recall, and F1 score for both accurate and resilient malware detection. Aurangzeb et al. employed a simple speaker-modeling approach with an accuracy of 95%. So then, knowing the truth, we have. that its precision, recall, and F1 score are around 90%, 92%, and 92%, and hence their model is not very useful in capturing the complex patterns that are embedded within the malware datasets [19]. In contrast, our proposed approach demonstrates significantly better 95.12% accuracy and balanced precision, recall and F1 score performance at 94.86%, 95.44%, and 95.14%, respectively, with a such that it may better classify. Casolare et al. achieved 84% accuracy using a shallow machine learning approach. We can calculate that their precision, recall, and F1 score are approximately 83.76% based on the accuracy. and as a result, their model struggles to identify the intricate patterns that lie behind the malware datasets [20]. McDonald et al. The blessing of the random forest worked as expected and gave 2,577 of the samples that were malicious (which is our true positive) and 1,698 for the benign which is our true negative. Nevertheless, random forest misclassified 547 samples as malicious false positives and 421 samples as benign false negatives. Random forest has a precision of 0.8249 recall at 0.8585, accuracy of 0.8153, F1-score 0.84188, and specificity 0.7563 of AUC [11]. Almomani et al. classified Android malware with an accuracy of 88.77% using 22 CNN models. Although they are competitive in accuracy, it is significantly less than the 95.12% of our suggested strategy. With this accuracy, we calculate their F1 score to be 86.5%, recall to be 86%, and precision to be 87%. Although their method uses deep learning to classify malware, our approach performs better than it in terms of accuracy, precision, and recall, suggesting that it detects malware more effectively and misclassifies less [21].

Table 4: Performance Comparison of Proposed Model with Existing Study

Study	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Aurangzeb et al. [15]	95	90	92	92
Casolare et al. [20]	84	83.76	-	-
McDonald et al. [11]	81.53	82.49	85.85	84.18
Almomani et al. [21]	88.77	87	86	86.5
Proposed Method	95.12	94.86	95.44	95.14

CHAPTER FIVE

CONCLUSIONS & FUTURE WORK

This chapter goes over the main results of the study, with a focus on how well image-based deep learning models can locate Android malware. The models worked well on the AndroDEX dataset, however there were certain problems, like the dataset being unbalanced, malware changing over time, and the high cost of computing. To fix these problems, future study might focus on adding real-time detection, using bigger and more varied datasets, and looking into lightweight models that can be used on mobile devices. Overall, the research provides a solid framework for improving smart, vision-based malware detection methods.

5.1 Conclusion

In this work, we have demonstrated deep learning approaches for detection of Android malware by using the AndroDEX dataset. The main goal was to measure the performance of different deep learning models for detection and categorization of malicious Android applications. We conducted a number of experiments on CNN, VGG16, MobileNetV2, ResNet50, InceptionV3, DenseNet121, and VGG19 to evaluate its various performances based on different evaluation metrics- Accuracy, Precision, Recall, F1 Score, AUC, FPR, FNR, and Training Time.

The result showed that the CNN approach generated great accuracy results that ranged 94.47% in average. It showed high performance in differentiating the benign and malicious applications, reflected in the high precision, recall, and F1 score. The VGG16 model also achieved good results with 95.12% Accuracy and comparable results in terms of Precision and F1 score and achieved high AUC values, showing the ability of the model to be successful also in the malware detection domain. MobileNetV2 on the other hand, slight less in performance also give satisfactory result with an average accuracy of 93.31% which indicates that it is suitable for resource-constraint condition which computational efficiency is being considered.

The best performing architectures that are more advanced such as ResNet50 and InceptionV3 had averaged accuracy lower than other architectures at 89.59% and 86.92% respectively. However, those models were still a good compromise between precision and recall and hence, could be especially interesting in sense that those showed highest robustness to the changes of data. DenseNet121 provided competitive results (89.60% average accuracy),

revealing its powerful feature extraction and complex pattern learning abilities of the network. The performance of VGG19 were the worst in terms of correct classification except in some specific cases with high recall values, they can be used to detect new malware samples. But it's higher False positive and False-negative rate indicates that a further fine-tuning or an alternative method is needed to enhance its detection precision. With regard to the computational efficiency, MobileNetV2 of an input size 224 indirectly appeared to also be favorable due to a shorter training time and so might be an option for real-time malware detection on resource- constrained environment. On the other hand, deeper models such as VGG16 and ResNet50, although they provide higher detection accuracy, they cost much more time for training, which may not be applicable for resource-constrained settings.

Taken as a whole, the tested DNN-based models were found effective for the identification of the Android malware. Some of the models were very good in terms of both accuracy and precision, but a few compromised statistically on computational performance. The use of alternate encoders or combined strategies hybrid or recurrent for a trade-off between detection performance and resource utilization may be interesting directions for future research. Moreover, deeper learning models especially those with deeper layers in Android malware detection, when combined with bigger data and real-time detection, could be more effective.

The results of this study extend existing knowledge in the field of deep learning applications in cybersecurity, providing a comprehensive assessment of the effectiveness and weaknesses of different architectures for Android malware detection.

5.2 Limitations

This study shows that image-based deep learning models are quite good at finding Android malware, but it's vital to note some drawbacks that could make it less useful in real life. These limits show where the current technique might not work as well as it could and provide ways to make it better.

1. The dataset might not be a good enough representation of new or very hidden malware, which would make the model less adaptable to real-world threats.
2. When we change bytecode into pictures, we could lose vital information about how things work or what they mean.

3. The deep learning models utilized are quite resource-intensive, which makes it hard to apply them in real time or on mobile devices.
4. Dynamic code loading and runtime obfuscation are two advanced evasion approaches that might work against the image-based static analysis method.
5. Class imbalance in the dataset can affect how well the model works, which could cause bias in categorization.

5.3 Future Work

Our research demonstrates that deep learning can help find Android malware, but there is still room for improvement. In the future, the focus will be on making real-time detection better, models more efficient, and models that can adapt to new threats.

1. *Integration of Real-Time Detection*: Future implementations will run on Android devices to detect and prevent malware in real-time, thereby enhancing mobile device security.
2. *Expansion to Zero-Day Threats*: Integrate some form of anomaly-based detection and generative models (GANs/transformers as an example) to detect zero-day and never-before seen malware samples.
3. *Explainable AI (XAI)*: Apply interpretable models to describe line and shape, enhancing user and developer confidence in detection systems.
4. *Lightweight Model Deployment*: Optimize and compress deep learning models (such as pruning, quantization) for running on memory-limited mobile devices without degradation of performance.
5. *Multimodal Analysis*: Fuse static (bytecode/image), dynamic (behavioral) and hybrid features to improve detection effectiveness and protect obfuscation robustness.
6. *Transfer Learning and Domain Adaptation*: Use domain adaptation methods to adapt models trained on existing datasets such as AndroDEX to new malware samples or to other datasets.
7. *Continuous Learning Framework*: Integrate online learning or incremental learning approaches which enable updating of the model once new malwares are detected in the future.
8. *Cross-Dataset Generalization Testing*: Test on other malware datasets such as CIC-InvesAndMal2019, Drebin, AMD to verify the generalization and robustness.

References

- [1] Y. L. Z. W. Y. X. Zhenlong Yuan, "Droid-Sec: Deep Learning in Android Malware Detection," in *Proceedings of the 2014 ACM conference on SIGCOMM*, 2014, August.
- [2] K. K. S. A. S. A. M. A. Pronoy Chopra, "A Deep Learning-Based Ensemble Framework for Robust Android Malware Detection," *IEEE Access*, vol. 13, pp. 46673 - 46696, 2025.
- [3] A. Kivva, "Mobile malware evolution in 2024," securelist by Kaspersky, 2025.
- [4] S. Q. A. A. S. S. A. A. A. M. u. H. Muhammad Umar Rashid, "Hybrid Android Malware Detection and Classification Using Deep Neural Networks," *International Journal of Computational Intelligence Systems*, vol. 18, no. 1, pp. 1-26, 2025.
- [5] G. P. G. S. K. P. K. Santosh K. Smmarwar, "An optimized and efficient android malware detection framework for future sustainable computing," *Sustainable Energy Technologies and Assessments*, 2022.
- [6] B. I. M. B. J. MÜLHEM İBRAHİM, "A Method for Automatic Android Malware Detection Based on Static Analysis and Deep Learning," *IEEE Access*, 2022.
- [7] A. Raval and M. Anwa, "Android Malware Detection: An Empirical Investigation into Machine Learning Classifiers," in *2024 IEEE International Conference on Information Reuse and Integration for Data Science (IRI)*, San Jose, CA, USA, 2024.
- [8] L. Taheri, A. F. A. Kadir and A. H. Lashkari, "Extensible Android Malware Detection and Family Classification Using Network-Flows and API-Calls," in *2019 International Carnahan Conference on Security Technology (ICCST)*, Chennai, India , 2019.
- [9] S. K. P. P. S. K. S. Vinayakumar R, "Detecting Android malware using Long Short-term Memory (LSTM)," *Journal of Intelligent & Fuzzy Systems: Applications in Engineering and Technology*, vol. 34, no. 3, pp. 1277-1288, 2018.
- [10] H. M. S. T. A. A. A. A. H. F. H Alshahrani, "DDefender: Android application threat detection using static and dynamic analysis," in *2018 IEEE International Conference on Consumer Electronics (ICCE)*, 2018.
- [11] N. H. W. B. G. R. K. B. J. Todd McDonald, "Machine Learning-Based Android Malware Detection Using Manifest Permissions," in *Proceedings of the 54th Hawaii International Conference on System Sciences*, 2021.

- [12] N. Aldhafferi, "Android Malware Detection Using Support Vector Regression for Dynamic Feature Analysis," *MDPI*, vol. 15, no. 1, p. 658, 2024.
- [13] P. C. Peter Barry, "Modern Embedded Computing: Designing Connected, Pervasive, Media-Rich Systems," The Netherlands, Elsevier, 2012.
- [14] "Services overview," 22 September 2022. [Online]. Available: <https://developer.android.com/develop/background-work/services>.
- [15] S. Aurangzeb, M. Aleem, M. T. Khan, G. Loukas and G. Sakellari, "Androdex_Images," *figshare. Dataset*, p. <https://doi.org/10.6084/m9.figshare.23931204.v1>, 2023.
- [16] O. C. Gael Varoquaux, "Evaluating Machine Learning Models and Their Diagnostic Value," *Machine Learning for Brain Disorders*, vol. 197, p. 601–630, 2023.
- [17] M. S. Tuan T. Nguyen, "Smart Camera Parking System With Auto Parking Spot Detection," *Computer Vision and Pattern Recognition*, 2024.
- [18] G. M. Ioannis Markoulidakis, "Probabilistic Confusion Matrix: A Novel Method for Machine Learning Algorithm Generalized Performance Analysis," *Technologies* , vol. 12, no. 7, p. 113, 2024.
- [19] M. A. M. T. K. G. L. & G. S. Sana Aurangzeb, "AndroDex: Android Dex Images of Obfuscated Malware," *Scientific Data*, vol. 11, no. 14, 2024.
- [20] G. C. G. I. ., F. M. ., F. M. A. S. ., M. T. Rosangela Casolare, "On the Resilience of Shallow Machine Learning Classification in Image-based Malware Detection," *Procedia Computer Science*, vol. 207, pp. 145-157, 2022.
- [21] M. A. W. E.-S. Iman Almomani, "Android malware analysis in a nutshell," *PLoS ONE*, vol. 17, no. 7, 2022.